

Spring PetClinic CI/CD Pipeline на Proxmox

Подробная инструкция по разворачиванию end-to-end CI/CD pipeline с GitLab, Kubernetes, Maven, Nexus, SonarQube на домашнем Proxmox сервере.

Архитектура решения

Компоненты инфраструктуры

- **GitLab CE** - система управления репозиториями и CI/CD
- **Kubernetes (K3s)** - оркестрация контейнеров (аналог DigitalOcean Kubernetes)
- **Nexus Repository** - хранилище артефактов
- **SonarQube** - анализ качества кода
- **GitLab Runner** - исполнитель CI/CD задач
- **HAProxy** - load balancer для доступа к сервисам

Сетевая топология

Internet (серый IP) → Router (10.0.10.1) → Proxmox (10.0.10.200)

↓

Внутренняя сеть 10.0.10.0/24	
GitLab VM:	10.0.10.10
K3s Master:	10.0.10.20
K3s Worker-1:	10.0.10.21
K3s Worker-2:	10.0.10.22
HAProxy LB:	10.0.10.30

Часть 1: Подготовка виртуальных машин

1.1 Требования к VM

VM	CPU	RAM	Disk	OS
GitLab	4	8GB	50GB	Ubuntu 22.04
K3s Master	2	4GB	40GB	Ubuntu 22.04
K3s Worker-1	2	8GB	60GB	Ubuntu 22.04
K3s Worker-2	2	8GB	60GB	Ubuntu 22.04
HAProxy	1	2GB	20GB	Ubuntu 22.04

1.2 Terraform конфигурация для Proxmox

Создайте файл `main.tf`:

```
hcl

terraform {
  required_providers {
    proxmox = {
      source = "telmate/proxmox"
      version = "2.9.14"
    }
  }
}

provider "proxmox" {
  pm_api_url      = "https://10.0.10.200:8006/api2/json"
  pm_user         = "root@pam"
  pm_password     = "your-password"
  pm_tls_insecure = true
}

# Cloud-init template (создайте заранее)
variable "template_name" {
  default = "ubuntu-2204-cloudinit-template"
}

# GitLab VM
resource "proxmox_vm_qemu" "gitlab" {
  name          = "gitlab"
  target_node   = "pve"
  clone         = var.template_name

  cores        = 4
  memory       = 8192
  scsihw       = "virtio-scsi-pci"

  disk {
    size      = "50G"
    type      = "scsi"
    storage   = "local-lvm"
  }

  network {
    model = "virtio"
    bridge = "vbr0"
  }

  ipconfig0 = "ip=10.0.10.10/24,gw=10.0.10.1"
  nameserver = "8.8.8.8"
}
```

```

ssh_user = "ubuntu"
sshkeys  = file("~/ssh/id_rsa.pub")
}

# K3s Master
resource "proxmox_vm_qemu" "k3s_master" {
  name          = "k3s-master"
  target_node   = "pve"
  clone         = var.template_name

  cores        = 2
  memory       = 4096

  disk {
    size      = "40G"
    type      = "scsi"
    storage   = "local-lvm"
  }

  network {
    model = "virtio"
    bridge = "vmbr0"
  }

  ipconfig0 = "ip=10.0.10.20/24,gw=10.0.10.1"
  nameserver = "8.8.8.8"

  ssh_user = "ubuntu"
  sshkeys  = file("~/ssh/id_rsa.pub")
}

# K3s Worker-1
resource "proxmox_vm_qemu" "k3s_worker1" {
  name          = "k3s-worker1"
  target_node   = "pve"
  clone         = var.template_name

  cores        = 2
  memory       = 8192

  disk {
    size      = "60G"
    type      = "scsi"
    storage   = "local-lvm"
  }

  network {
    model = "virtio"
    bridge = "vmbr0"
  }

```

```

}

ipconfig0 = "ip=10.0.10.21/24,gw=10.0.10.1"
nameserver = "8.8.8.8"

ssh_user = "ubuntu"
sshkeys = file("~/ssh/id_rsa.pub")
}

# K3s Worker-2
resource "proxmox_vm_qemu" "k3s_worker2" {
  name          = "k3s-worker2"
  target_node   = "pve"
  clone         = var.template_name

  cores        = 2
  memory       = 8192

  disk {
    size      = "60G"
    type      = "scsi"
    storage   = "local-lvm"
  }

  network {
    model = "virtio"
    bridge = "vmbr0"
  }

  ipconfig0 = "ip=10.0.10.22/24,gw=10.0.10.1"
  nameserver = "8.8.8.8"

  ssh_user = "ubuntu"
  sshkeys = file("~/ssh/id_rsa.pub")
}

# HAProxy LB
resource "proxmox_vm_qemu" "haproxy" {
  name          = "haproxy"
  target_node   = "pve"
  clone         = var.template_name

  cores        = 1
  memory       = 2048

  disk {
    size      = "20G"
    type      = "scsi"
    storage   = "local-lvm"
  }

```

```

}

network {
  model = "virtio"
  bridge = "vmbr0"
}

ipconfig0 = "ip=10.0.10.30/24,gw=10.0.10.1"
nameserver = "8.8.8.8"

ssh_user = "ubuntu"
sshkeys = file("~/ssh/id_rsa.pub")
}

output "vm_ips" {
  value = {
    gitlab      = "10.0.10.10"
    k3s_master  = "10.0.10.20"
    k3s_worker1 = "10.0.10.21"
    k3s_worker2 = "10.0.10.22"
    haproxy     = "10.0.10.30"
  }
}

```

1.3 Создание Cloud-Init шаблона

На Proxmox хосте:

```

bash

# Скачать Ubuntu Cloud Image
cd /var/lib/vz/template/iso
wget https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img

# Создать VM для шаблона
qm create 9000 --name ubuntu-2204-cloudinit-template --memory 2048 --net0 virtio,bridge=v
qm importdisk 9000 jammy-server-cloudimg-amd64.img local-lvm
qm set 9000 --scsihw virtio-scsi-pci --scsi0 local-lvm:vm-9000-disk-0
qm set 9000 --ide2 local-lvm:cloudinit
qm set 9000 --boot c --bootdisk scsi0
qm set 9000 --serial0 socket --vga serial0
qm set 9000 --agent enabled=1

# Конвертировать в шаблон
qm template 9000

```

1.4 Развертывание VM через Terraform

На Windows машине:

```
bash

# Инициализация
terraform init

# Планирование
terraform plan

# Применение
terraform apply -auto-approve

# Проверка доступности
ping 10.0.10.10
ping 10.0.10.20
```

Часть 2: Установка GitLab CE

2.1 Подключение к GitLab VM

```
bash

ssh ubuntu@10.0.10.10
```

2.2 Установка GitLab

```
bash

# Обновление системы
sudo apt update && sudo apt upgrade -y

# Установка зависимостей
sudo apt install -y curl openssh-server ca-certificates tzdata perl

# Добавление репозитория GitLab
curl https://packages.gitlab.com/install/repositories/gitlab/gitlab-ce/script.deb.sh | sudo sh

# Установка GitLab CE
sudo EXTERNAL_URL="http://10.0.10.10" apt install gitlab-ce

# Получение пароля root
sudo cat /etc/gitlab/initial_root_password
```

2.3 Настройка GitLab

```
bash
```

```
# Редактирование конфигурации
sudo nano /etc/gitlab/gitlab.rb
```

Измените следующие параметры:

```
ruby

external_url 'http://gitlab.local.lab'
gitlab_rails['gitlab_shell_ssh_port'] = 22
gitlab_rails['time_zone'] = 'Asia/Almaty'

# Настройка памяти
puma['worker_processes'] = 2
sidekiq['max_concurrency'] = 10

# Отключение неиспользуемых компонентов
prometheus_monitoring['enable'] = false
grafana['enable'] = false
```

Примените изменения:

```
bash

sudo gitlab-ctl reconfigure
sudo gitlab-ctl restart
...

### 2.4 Настройка hosts на Windows

**Запустите Notepad от имени администратора:**
...
C:\Windows\System32\drivers\etc\hosts
...

Добавьте записи:
...

10.0.10.10    gitlab.local.lab
10.0.10.30    petclinic.local.lab
10.0.10.30    nexus.local.lab
10.0.10.30    sonarqube.local.lab
```

Часть 3: Установка K3s Kubernetes

3.1 Установка K3s Master

```
bash
```

```
ssh ubuntu@10.0.10.20

# Установка K3s
curl -sfL https://get.k3s.io | sh -s - server \
  --disable traefik \
  --disable servicelb \
  --node-ip 10.0.10.20 \
  --node-external-ip 10.0.10.20

# Получение токена для worker нод
sudo cat /var/lib/rancher/k3s/server/node-token

# Копирование kubeconfig
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown ubuntu:ubuntu ~/.kube/config

# Проверка
kubectl get nodes
```

3.2 Установка K3s Workers

Worker-1:

```
bash

ssh ubuntu@10.0.10.21

# Установка (замените TOKEN на значение из master)
curl -sfL https://get.k3s.io | K3S_URL=https://10.0.10.20:6443 \
  K3S_TOKEN="YOUR_TOKEN_HERE" sh -
```

Worker-2:

```
bash

ssh ubuntu@10.0.10.22

# Установка
curl -sfL https://get.k3s.io | K3S_URL=https://10.0.10.20:6443 \
  K3S_TOKEN="YOUR_TOKEN_HERE" sh -
```

Проверка на Master:

```
bash
```



```
kubectl get nodes
# Должны появиться все 3 ноды
```

3.3 Установка MetalLB (LoadBalancer)

```
bash

# На K3s Master
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.13.12/config/manifests/kustomize.yaml

# Дождитесь готовности подов
kubectl wait --namespace metallb-system \
  --for=condition=ready pod \
  --selector=app=metallb \
  --timeout=90s
```

Создайте файл `metallb-config.yaml`:

```
yaml

apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: default
  namespace: metallb-system
spec:
  addresses:
  - 10.0.10.100-10.0.10.150
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: default
  namespace: metallb-system
spec:
  ipAddressPools:
  - default
```

Примените:

```
bash

kubectl apply -f metallb-config.yaml
```

Часть 4: Установка Helm

4.1 На K3s Master

```
bash
```

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

```
# Проверка  
helm version
```

4.2 На Windows машине

Скачайте Helm для Windows: <https://github.com/helm/helm/releases>

Распакуйте и добавьте в PATH, либо через Chocolatey:

```
powershell
```

```
choco install kubernetes-helm
```

4.3 Копирование kubeconfig на Windows

На K3s Master:

```
bash
```

```
cat ~/.kube/config
```

На Windows:

Создайте файл `C:\Users\YourUser\.kube\config` и вставьте содержимое, заменив:

```
yaml
```

```
server: https://127.0.0.1:6443
```

на:

```
yaml
```

```
server: https://10.0.10.20:6443
```

Проверьте подключение:

```
powershell
```

```
kubectl get nodes
```

Часть 5: Установка SonarQube

5.1 Добавление Helm репозитория

```
bash

ssh ubuntu@10.0.10.20

helm repo add sonarqube https://SonarSource.github.io/helm-chart-sonarqube
helm repo update
```

5.2 Подготовка конфигурации

Создайте `sonarqube-values.yaml`:

```
yaml

service:
  type: LoadBalancer
  loadBalancerIP: 10.0.10.101

resources:
  requests:
    cpu: 500m
    memory: 2Gi
  limits:
    cpu: 2000m
    memory: 4Gi

persistence:
  enabled: true
  storageClass: "local-path"
  size: 20Gi

postgresql:
  enabled: true
  persistence:
    enabled: true
    size: 10Gi
```

5.3 Установка

```
bash

kubectl create namespace sonarqube

helm install sonarqube sonarqube/sonarqube \
```

```
--namespace sonarqube \  
-f sonarqube-values.yaml
```

Проверка

```
kubectl get pods -n sonarqube  
kubectl get svc -n sonarqube
```

5.4 Получение пароля

Дождитесь запуска (может занять 5-10 минут):

```
bash  
  
kubectl wait --for=condition=ready pod \  
-l app=sonarqube \  
-n sonarqube \  
--timeout=600s
```

Доступ:

- URL: <http://10.0.10.101:9000>
- Login: admin
- Password: admin (измените при первом входе)

Часть 6: Установка Nexus Repository

6.1 Добавление репозитория

```
bash  
  
helm repo add sonatype https://sonatype.github.io/helm3-charts/  
helm repo update
```

6.2 Подготовка конфигурации

Создайте `nexus-values.yaml`:

```
yaml  
  
service:  
  type: LoadBalancer  
  loadBalancerIP: 10.0.10.102  
  
resources:  
  requests:  
    cpu: 500m  
    memory: 2Gi
```

```
limits:
  cpu: 2000m
  memory: 4Gi

persistence:
  enabled: true
  storageClass: "local-path"
  storageSize: 50Gi

nexus:
  env:
    - name: INSTALL4J_ADD_VM_PARAMS
      value: "-Xms1200M -Xmx1200M -XX:MaxDirectMemorySize=2G"
```

6.3 Установка

```
bash

kubectl create namespace nexus

helm install nexus sonatype/nexus-repository-manager \
  --namespace nexus \
  -f nexus-values.yaml

# Проверка
kubectl get pods -n nexus
kubectl get svc -n nexus
```

6.4 Получение пароля

```
bash

# Дождитесь готовности
kubectl wait --for=condition=ready pod \
  -l app=nexus-repository-manager \
  -n nexus \
  --timeout=600s

# Получите имя пода
POD_NAME=$(kubectl get pods -n nexus -l app=nexus-repository-manager -o jsonpath='{.items

# Получите пароль
kubectl exec -n nexus $POD_NAME -- cat /nexus-data/admin.password
```

Доступ:

- URL: <http://10.0.10.102:8081>

- Login: admin
- Password: (из команды выше)

Часть 7: Настройка HAProxy

7.1 Установка HAProxy

```
bash

ssh ubuntu@10.0.10.30

sudo apt update
sudo apt install -y haproxy
```

7.2 Конфигурация

```
bash

sudo nano /etc/haproxy/haproxy.cfg
```

Добавьте в конец файла:

```
haproxy

frontend http_front
    bind *:80
    mode http

    acl is_nexus hdr(host) -i nexus.local.lab
    acl is_sonar hdr(host) -i sonarqube.local.lab
    acl is_petclinic hdr(host) -i petclinic.local.lab

    use_backend nexus_back if is_nexus
    use_backend sonar_back if is_sonar
    use_backend petclinic_back if is_petclinic

backend nexus_back
    mode http
    balance roundrobin
    server nexus 10.0.10.102:8081 check

backend sonar_back
    mode http
    balance roundrobin
    server sonar 10.0.10.101:9000 check

backend petclinic_back
```

```
mode http
balance roundrobin
server petclinic 10.0.10.103:80 check
```

Перезапустите HAProxy:

```
bash

sudo systemctl restart haproxy
sudo systemctl enable haproxy
sudo systemctl status haproxy
```

Часть 8: Настройка Nexus Repository

8.1 Вход в Nexus

Откройте <http://nexus.local.lab> в браузере, войдите с admin и паролем.

8.2 Создание Maven репозиториев

1. Перейдите в **Settings** → **Repositories** → **Create repository**
2. Создайте **maven2 (hosted)** с именем `maven-hosted-release`:
 - Version policy: Release
 - Deployment policy: Disable redeploy
3. Создайте **maven2 (hosted)** с именем `maven-hosted-snapshot`:
 - Version policy: Snapshot
 - Deployment policy: Allow redeploy

8.3 Создание Docker репозитория (опционально)

1. **Create repository** → **docker (hosted)**
2. Name: `docker-hosted`
3. HTTP: 8082
4. Enable Docker V1 API: ✓

Часть 9: Настройка SonarQube

9.1 Вход в SonarQube

Откройте <http://sonarqube.local.lab>, войдите (admin/admin), смените пароль.

9.2 Создание проекта

1. Нажмите **Create Project** → **Manually**

2. Project key: `spring-petclinic`
3. Display name: `Spring PetClinic`
4. Нажмите **Set Up**

9.3 Генерация токена

1. Выберите **Locally**
2. Generate token: `gitlab-ci`
3. Validity: 30 days
4. **Generate** и скопируйте токен

9.4 Создание Quality Gate (опционально)

1. **Quality Gates** → **Create**
2. Name: `GitLab CI Gate`
3. Добавьте условия:
 - Coverage < 80% (Warning)
 - Bugs > 0 (Error)
 - Code Smells > 5 (Warning)

Часть 10: Настройка GitLab CI/CD

10.1 Создание проекта в GitLab

1. Откройте <http://gitlab.local.lab>
2. Создайте новый проект: `spring-petclinic`
3. Visibility: Public

10.2 Настройка CI/CD переменных

Перейдите в **Settings** → **CI/CD** → **Variables**:

Key	Value	Masked	Protected
<code>NEXUS_USER</code>	admin	No	No
<code>NEXUS_PASSWORD</code>	(ваш пароль)	Yes	No
<code>SONAR_HOST_URL</code>	http://10.0.10.101:9000	No	No
<code>SONAR_TOKEN</code>	(токен из SonarQube)	Yes	No
<code>CI_REGISTRY</code>	https://index.docker.io/v1/	No	No
<code>CI_REGISTRY_USER</code>	(Docker Hub username)	No	No
<code>CI_REGISTRY_PASSWORD</code>	(Docker Hub token)	Yes	No

Key	Value	Masked	Protected
KUBECONFIG	(содержимое ~/.kube/config)	No	No

Для KUBECONFIG:

- Type: **File**
- Value: Содержимое kubeconfig с master ноды

10.3 Клонирование проекта

На Windows:

```
bash
```

```
git clone https://github.com/spring-projects/spring-petclinic.git
cd spring-petclinic
```

10.4 Изменение pom.xml

Добавьте в <properties>:

```
xml
```

```
<nexus.host.url>http://10.0.10.102:8081</nexus.host.url>
```

Добавьте после </repositories>:

```
xml
```

```
<distributionManagement>
  <repository>
    <id>nexus</id>
    <name>Nexus Release Repository</name>
    <url>${nexus.host.url}/repository/maven-hosted-release/</url>
  </repository>
  <snapshotRepository>
    <id>nexus</id>
    <name>Nexus Snapshot Repository</name>
    <url>${nexus.host.url}/repository/maven-hosted-snapshot/</url>
  </snapshotRepository>
</distributionManagement>
```

10.5 Создание Maven settings.xml

Создайте .m2/settings.xml:

```
xml
```

```
<settings>
  <servers>
    <server>
      <id>nexus</id>
      <username>${env.NEXUS_USER}</username>
      <password>${env.NEXUS_PASSWORD}</password>
    </server>
  </servers>
</settings>
```

10.6 Создание sonar-project.properties

properties

```
sonar.projectKey=spring-petclinic
sonar.projectName=spring-petclinic
sonar.projectVersion=1.0
sonar.sources=src/main
sonar.tests=src/test
sonar.java.binaries=target/classes
sonar.language=java
sonar.sourceEncoding=UTF-8
sonar.java.libraries=target/classes
```

10.7 Создание Dockerfile

dockerfile

```
FROM openjdk:8-jre-alpine
EXPOSE 8080
COPY target/*.war /usr/bin/spring-petclinic.war
ENTRYPOINT ["java", "-jar", "/usr/bin/spring-petclinic.war", "--server.port=8080"]
```

10.8 Создание Helm Chart

bash

```
mkdir -p petclinic-chart/templates
```

petclinic-chart/Chart.yaml:

yaml

```
apiVersion: v2
name: petclinic
```

```
description: Spring PetClinic Application
version: 1.0.0
appVersion: "1.0"
```

petclinic-chart/values.yaml:

```
yaml

image:
  repository: yourdockerhubuser/spring-petclinic
  tag: latest
  pullPolicy: Always

service:
  type: LoadBalancer
  loadBalancerIP: 10.0.10.103
  port: 80
  targetPort: 8080

replicaCount: 1
```

petclinic-chart/templates/deployment.yaml:

```
yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Chart.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Chart.Name }}
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: {{ .Values.service.targetPort }}
```

petclinic-chart/templates/service.yaml:

yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Chart.Name }}
spec:
  type: {{ .Values.service.type }}
  loadBalancerIP: {{ .Values.service.loadBalancerIP }}
  selector:
    app: {{ .Chart.Name }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}
```

10.9 Создание .gitlab-ci.yml

Создайте `.gitlab-ci.yml` в корне проекта:

yaml

```
variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository"
  M2_EXTRA: "-s .m2/settings.xml"
  IMAGE_NAME: "spring-petclinic"
  TAG: "$CI_COMMIT_SHA"

image: maven:3.8.2-openjdk-11

stages:
  - build
  - test
  - quality
  - package
  - dockerize
  - deploy

cache:
  paths:
    - .m2/repository

build_job:
  stage: build
  script:
    - echo "Building application..."
    - mvn $M2_EXTRA clean compile
  artifacts:
    paths:
```

- target/

test_job:

stage: test

script:

- echo "Running tests..."
- mvn \$M2_EXTRA test

dependencies:

- build_job

sonar_scan:

stage: quality

image:

name: sonarsource/sonar-scanner-cli:latest
entrypoint: [""]

variables:

SONAR_USER_HOME: "\${CI_PROJECT_DIR}/.sonar"
GIT_DEPTH: "0"

script:

- sonar-scanner -Dsonar.qualitygate.wait=true

dependencies:

- build_job

allow_failure: false

package_job:

stage: package

script:

- echo "Packaging application..."
- mvn \$M2_EXTRA package -DskipTests
- ls -l target/*.war

artifacts:

paths:

- target/*.war

expire_in: 1 week

dependencies:

- build_job

push_to_nexus:

stage: package

script:

- echo "Deploying to Nexus..."
- mvn \$M2_EXTRA deploy -DskipTests

dependencies:

- package_job

only:

- main
- master

dockerize:

```

stage: dockerize
image:
  name: gcr.io/kaniko-project/executor:v1.9.0-debug
  entrypoint: [""]
script:
  - mkdir -p /kaniko/.docker
  - echo "{\"auths\":{\"${CI_REGISTRY}\":{\"auth\":\"${printf \"%s:%s\" \"${CI_REGISTRY_USI
  - /kaniko/executor
    --context "${CI_PROJECT_DIR}"
    --dockerfile "${CI_PROJECT_DIR}/Dockerfile"
    --destination "${CI_REGISTRY_USER}/${IMAGE_NAME}:${TAG}"
    --destination "${CI_REGISTRY_USER}/${IMAGE_NAME}:latest"
dependencies:
  - package_job
only:
  - main
  - master

deploy_to_k8s:
stage: deploy
image: kunchalavikram/kubectl_helm_cli:latest
script:
  - mkdir -p ~/.kube
  - cat $KUBECONFIG > ~/.kube/config
  - helm upgrade --install petclinic ./petclinic-chart
    --set image.tag=${TAG}
    --set image.repository=${CI_REGISTRY_USER}/${IMAGE_NAME}
    --namespace default
environment:
  name: production
  url: http://petclinic.local.lab
only:
  - main
  - master
when: manual

```

10.10 Отправка кода в GitLab

```

bash

git remote remove origin
git remote add origin http://gitlab.local.lab/root/spring-petclinic.git
git add .
git commit -m

```

Continue

```
bash
```

```
git commit -m "Initial commit with CI/CD pipeline"  
git push -u origin master
```

Часть 11: Установка GitLab Runner в Kubernetes

11.1 Добавление Helm репозитория GitLab

```
bash
```

```
ssh ubuntu@10.0.10.20
```

```
helm repo add gitlab https://charts.gitlab.io  
helm repo update
```

11.2 Получение Registration Token

1. Откройте GitLab: <http://gitlab.local.lab>
2. Перейдите в **Admin Area** (гаечный ключ) → **CI/CD** → **Runners**
3. Скопируйте **Registration token**

11.3 Подготовка values для Runner

Создайте `gitlab-runner-values.yaml`:

```
yaml
```

```
gitlabUrl: http://10.0.10.10/  
runnerRegistrationToken: "YOUR_REGISTRATION_TOKEN_HERE"  
  
rbac:  
  create: true  
  
runners:  
  config: |  
    [[runners]]  
    [runners.kubernetes]  
      namespace = "{{.Release.Namespace}}"  
      image = "ubuntu:22.04"  
      privileged = true  
    [[runners.kubernetes.volumes.empty_dir]]  
      name = "docker-certs"  
      mount_path = "/certs/client"  
      medium = "Memory"  
  
tags: "k8s,kubernetes"
```

```
runUntagged: true
locked: false

resources:
  limits:
    memory: 256Mi
    cpu: 200m
  requests:
    memory: 128Mi
    cpu: 100m
```

11.4 Установка Runner

```
bash

kubectl create namespace gitlab-runner

helm install gitlab-runner gitlab/gitlab-runner \
  --namespace gitlab-runner \
  -f gitlab-runner-values.yaml

# Проверка
kubectl get pods -n gitlab-runner
kubectl logs -n gitlab-runner -l app=gitlab-runner-gitlab-runner
```

11.5 Проверка регистрации

Вернитесь в GitLab **Admin Area** → **CI/CD** → **Runners**. Вы должны увидеть новый Runner с тегом `k8s`.

Настройка Runner:

1. Нажмите на Runner
2. Установите:
 - ✓ Run untagged jobs
 - ✓ Lock to current projects (снимите)
 - Maximum job timeout: 3600

Часть 12: Адаптация Pipeline для Kubernetes Runner

12.1 Обновление `.gitlab-ci.yml` для использования Runner

Измените `.gitlab-ci.yml`:

```
yaml
```



```
variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository"
  M2_EXTRA: "-s .m2/settings.xml"
  IMAGE_NAME: "spring-petclinic"
  TAG: "$CI_COMMIT_SHA"

default:
  tags:
    - k8s

image: maven:3.8.2-openjdk-11

stages:
  - build
  - test
  - quality
  - package
  - dockerize
  - deploy

cache:
  paths:
    - .m2/repository

build_job:
  stage: build
  script:
    - echo "Building application..."
    - mvn $M2_EXTRA clean compile
  artifacts:
    paths:
      - target/
    expire_in: 1 hour

test_job:
  stage: test
  script:
    - echo "Running tests..."
    - mvn $M2_EXTRA test
  artifacts:
    reports:
      junit:
        - target/surefire-reports/TEST-*.xml
  dependencies:
    - build_job

sonar_scan:
  stage: quality
  image:
```

```

    name: sonarsource/sonar-scanner-cli:latest
    entrypoint: [""]
variables:
  SONAR_USER_HOME: "${CI_PROJECT_DIR}/.sonar"
  GIT_DEPTH: "0"
script:
  - sonar-scanner
    -Dsonar.projectKey=spring-petclinic
    -Dsonar.sources=src/main
    -Dsonar.java.binaries=target/classes
    -Dsonar.host.url=${SONAR_HOST_URL}
    -Dsonar.login=${SONAR_TOKEN}
    -Dsonar.qualitygate.wait=true
dependencies:
  - build_job
allow_failure: true

package_job:
  stage: package
  script:
    - echo "Packaging application..."
    - mvn $M2_EXTRA package -DskipTests
    - ls -lh target/*.war
  artifacts:
    paths:
      - target/*.war
    expire_in: 1 week
  dependencies:
    - build_job

push_to_nexus:
  stage: package
  script:
    - echo "Deploying artifacts to Nexus..."
    - mvn $M2_EXTRA deploy -DskipTests
  dependencies:
    - package_job
  only:
    - main
    - master

dockerize:
  stage: dockerize
  image:
    name: gcr.io/kaniko-project/executor:v1.9.0-debug
    entrypoint: [""]
  before_script:
    - mkdir -p /kaniko/.docker
    - echo "{\"auths\":{\"${CI_REGISTRY}\":{\"auth\":\"$(printf \"%s:%s\" \"${CI_REGISTRY_USI

```

script:

- echo "Building Docker image..."
- /kaniko/executor
- context "\${CI_PROJECT_DIR}"
- dockerfile "\${CI_PROJECT_DIR}/Dockerfile"
- destination "\${CI_REGISTRY_USER}/\${IMAGE_NAME}:\${TAG}"
- destination "\${CI_REGISTRY_USER}/\${IMAGE_NAME}:latest"
- cache=true
- cache-ttl=24h

dependencies:

- package_job

only:

- main
- master

deploy_to_k8s:

stage: deploy

image:

name: alpine/helm:latest
entrypoint: [""]

before_script:

- apk add --no-cache curl
- curl -LO "https://dl.k8s.io/release/v1.28.0/bin/linux/amd64/kubectl"
- chmod +x kubectl
- mv kubectl /usr/local/bin/
- mkdir -p ~/.kube
- cat \$KUBECONFIG > ~/.kube/config
- chmod 600 ~/.kube/config

script:

- echo "Deploying to Kubernetes..."
- helm version
- kubectl version --client
- kubectl cluster-info
- |
 helm upgrade --install petclinic ./petclinic-chart \
 --set image.tag=\${TAG} \
 --set image.repository=\${CI_REGISTRY_USER}/\${IMAGE_NAME} \
 --namespace default \
 --wait \
 --timeout 5m
- kubectl get pods -l app=petclinic
- kubectl get svc petclinic

environment:

name: production
url: http://petclinic.local.lab
on_stop: stop_deployment

only:

- main
- master

```
when: manual

stop_deployment:
  stage: deploy
  image: alpine/helm:latest
  script:
    - helm uninstall petclinic --namespace default || true
  environment:
    name: production
    action: stop
  when: manual
  only:
    - main
    - master
```

12.2 Отправка изменений

```
bash

git add .gitlab-ci.yml
git commit -m "Update pipeline for Kubernetes Runner"
git push origin master
```

Часть 13: Запуск и проверка Pipeline

13.1 Мониторинг Pipeline

1. Откройте GitLab: <http://gitlab.local.lab/root/spring-petclinic>
2. Перейдите в **CI/CD → Pipelines**
3. Должен запуститься новый pipeline

13.2 Проверка выполнения каждого этапа

Build Stage:

```
bash

# Проверка подов runner
kubectl get pods -n gitlab-runner

# Логи джоба
kubectl logs -n gitlab-runner -l app=gitlab-runner-gitlab-runner -f
```

Test Stage:

- Проверьте, что тесты прошли успешно

- Просмотрите JUnit отчеты в GitLab

Quality Stage:

- Откройте SonarQube: <http://sonarqube.local.lab>
- Проверьте результаты анализа проекта `spring-petclinic`

Package Stage:

- Проверьте артефакты в GitLab (справа от джоба кнопка Download)

Push to Nexus:

- Откройте Nexus: <http://nexus.local.lab>
- **Browse** → `maven-hosted-release`
- Проверьте наличие `org/springframework/samples/petclinic/spring-petclinic/1.0/`

Dockerize Stage:

- Проверьте Docker Hub: <https://hub.docker.com/>
- Образ должен появиться в репозитории

Deploy Stage:

- Это ручной этап, нажмите кнопку **Play** (▶)
- После завершения:

```
bash

kubectl get pods
kubectl get svc petclinic

# Должны увидеть LoadBalancer IP
NAME          TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
petclinic     LoadBalancer  10.43.x.x        10.0.10.103      80:xxxxx/TCP     2m
```

13.3 Проверка приложения

Откройте браузер: <http://petclinic.local.lab>

Вы должны увидеть работающее приложение Spring PetClinic.

Часть 14: Troubleshooting

14.1 Pipeline не запускается

```
bash
```

```
# Проверка Runner
kubectl get pods -n gitlab-runner
kubectl logs -n gitlab-runner -l app=gitlab-runner-gitlab-runner

# В GitLab проверьте Admin Area → Runners
# Runner должен быть зеленым (online)
```

14.2 Maven build fails

```
bash

# Проверьте настройки Nexus в pom.xml
# Проверьте переменные NEXUS_USER и NEXUS_PASSWORD в GitLab
```

14.3 SonarQube scan fails

```
bash

# Проверьте доступность SonarQube
curl http://10.0.10.101:9000/api/system/status

# Проверьте SONAR_TOKEN в GitLab Variables
# Проверьте sonar-project.properties
```

14.4 Docker push fails

```
bash

# Проверьте Docker Hub credentials
# Убедитесь, что CI_REGISTRY_USER и CI_REGISTRY_PASSWORD корректны
# Проверьте наличие репозитория на Docker Hub
```

14.5 Kubernetes deployment fails

```
bash

# Проверьте kubernetes
kubectl get nodes

# Проверьте namespace
kubectl get ns

# Проверьте логи пода
kubectl logs -l app=petsclinic
```

```
# Проверьте события
kubectl get events --sort-by='.lastTimestamp'
```

14.6 LoadBalancer IP не назначается

```
bash

# Проверьте MetalLB
kubectl get pods -n metallb-system

# Проверьте IPAddressPool
kubectl get ipaddresspools -n metallb-system

# Проверьте логи MetalLB
kubectl logs -n metallb-system -l app=metallb
```

Часть 15: Оптимизация и улучшения

15.1 Кэширование Maven зависимостей

Добавьте в `.gitlab-ci.yml`:

```
yaml

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - .m2/repository
  policy: pull-push
```

15.2 Параллельное выполнение тестов

```
yaml

test_job:
  stage: test
  parallel: 3
  script:
    - mvn $M2_EXTRA test -Dtest=**/*Test.java
```

15.3 Автоматический Rollback

Добавьте в `.gitlab-ci.yml`:

```
yaml
```

```
rollback:
  stage: deploy
  image: alpine/helm:latest
  script:
    - helm rollback petclinic
  when: manual
  only:
    - main
    - master
```

15.4 Monitoring с Prometheus

```
bash

# Установка Prometheus
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace
```

15.5 Настройка TLS для GitLab

Для production использования настройте Let's Encrypt или self-signed сертификаты.

Часть 16: Backup и восстановление

16.1 Backup GitLab

```
bash

ssh ubuntu@10.0.10.10

# Создание backup
sudo gitlab-backup create

# Backup сохраняется в /var/opt/gitlab/backups/
```

16.2 Backup Kubernetes PVCs

```
bash

# Для SonarQube
kubectl get pvc -n sonarqube
kubectl exec -n sonarqube POD_NAME -- tar czf /tmp/backup.tar.gz /opt/sonarqube/data
```



```
# Копирование backup
kubectl cp sonarqube/POD_NAME:/tmp/backup.tar.gz ./sonarqube-backup.tar.gz
```

16.3 Backup Nexus

```
bash

# Для Nexus
kubectl get pvc -n nexus
kubectl exec -n nexus POD_NAME -- tar czf /tmp/backup.tar.gz /nexus-data

kubectl cp nexus/POD_NAME:/tmp/backup.tar.gz ./nexus-backup.tar.gz
```

Часть 17: Полезные команды

17.1 GitLab

```
bash

# Рестарт GitLab
sudo gitlab-ctl restart

# Проверка статуса
sudo gitlab-ctl status

# Логи
sudo gitlab-ctl tail
```

17.2 Kubernetes

```
bash

# Проверка всех ресурсов
kubectl get all -A

# Описание пода
kubectl describe pod POD_NAME

# Exec в под
kubectl exec -it POD_NAME -- bash

# Port-forward для отладки
kubectl port-forward svc/SERVICE_NAME 8080:80

# Просмотр логов
kubectl logs -f POD_NAME
```

```
# Удаление застрявших подов
kubectl delete pod POD_NAME --grace-period=0 --force
```

17.3 Helm

```
bash

# Список релизов
helm list -A

# История релиза
helm history RELEASE_NAME

# Откат
helm rollback RELEASE_NAME REVISION

# Удаление релиза
helm uninstall RELEASE_NAME -n NAMESPACE
```

Часть 18: Дополнительные возможности

18.1 Multi-branch Pipeline

Добавьте в `.gitlab-ci.yml`:

```
yaml

deploy_dev:
  stage: deploy
  script:
    - helm upgrade --install petclinic-dev ./petclinic-chart
    --namespace dev
    --create-namespace
  environment:
    name: development
  only:
    - develop

deploy_prod:
  stage: deploy
  script:
    - helm upgrade --install petclinic ./petclinic-chart
    --namespace production
  environment:
    name: production
  only:
    - master
  when: manual
```

18.2 Slack уведомления

Добавьте в конец `.gitlab-ci.yml`:

```
yaml

notify_slack:
  stage: .post
  script:
    - 'curl -X POST -H "Content-type: application/json"
      --data "{\"text\": \"Pipeline ${CI_PIPELINE_ID} completed with status: ${CI_JOB_STATU}
      ${SLACK_WEBHOOK_URL}'
  when: always
```

18.3 Scheduled Pipelines

В GitLab: **CI/CD** → **Schedules** → **New schedule**

Заключение

Вы развернули полноценную CI/CD инфраструктуру на домашнем Proxmox сервере, включающую:

- ✓ GitLab CE для управления кодом и CI/CD
- ✓ Kubernetes кластер (K3s) для оркестрации
- ✓ Nexus Repository для артефактов
- ✓ SonarQube для анализа качества кода
- ✓ GitLab Runner в Kubernetes
- ✓ HAProxy для балансировки нагрузки
- ✓ Полный CI/CD pipeline с автоматическими тестами, анализом и деплоем

Следующие шаги

1. Настройте мониторинг (Prometheus + Grafana)
2. Добавьте логирование (ELK Stack)
3. Настройте автоматические backups
4. Реализуйте Blue-Green или Canary deployment
5. Добавьте security scanning (Trivy, OWASP Dependency Check)

Полезные ссылки

- GitLab CI/CD Documentation: <https://docs.gitlab.com/ee/ci/>
- K3s Documentation: <https://docs.k3s.io/>

- Helm Documentation: <https://helm.sh/docs/>
- Nexus Documentation: <https://help.sonatype.com/>
- SonarQube Documentation: <https://docs.sonarqube.org/>